

Implementación de un Prototipo Interactivo de Alta Fidelidad (Pixel Perfect e Interacción)

Un **prototipo de alta fidelidad** – también llamado *diseño Pixel Perfect* – es una representación casi exacta del producto final, tanto visualmente como en comportamiento. A diferencia de los wireframes o prototipos de baja/mediana fidelidad, un diseño Pixel Perfect incluye todos los detalles de la interfaz, con cada elemento meticulosamente ubicado según las especificaciones de diseño, sin desalineaciones ni inconsistencias ¹ ². Esto significa incorporar la paleta de colores final, tipografías precisas, iconografía e imágenes reales o de alta calidad, así como espaciados y dimensiones exactos de cada componente ³. El resultado es una **experiencia visual completa y realista**, fiel a la identidad de marca, que simula incluso animaciones, transiciones y microinteracciones tal como se espera en el producto final ⁴. Un diseño de alta fidelidad Pixel Perfect permite **validar la apariencia y comportamiento** antes del desarrollo, sirviendo de plano detallado para los desarrolladores y asegurando que el producto entregado cumpla con los estándares profesionales de la industria ⁵. A continuación, profundizamos en los principios, técnicas y consideraciones clave para lograr un prototipo interactivo de alta fidelidad exitoso, así como en aspectos de diseño de interacción en distintos dispositivos.

Diseño Pixel Perfect y Principios de Alta Fidelidad

El enfoque *Pixel Perfect* se basa en alcanzar la máxima fidelidad visual: cada píxel en la pantalla cuenta. **Principios clave** incluyen la consistencia visual, precisión en la alineación y detalle exhaustivo en todos los elementos de la interfaz. En la práctica, esto se traduce en adherirse a guías de estilo definidas (colores exactos, fuentes específicas, tamaños uniformes), mantener una distribución equilibrada y asegurarse de que todos los componentes estén alineados en una retícula subyacente. Un principio fundamental es “*obsesionarse con los detalles de la UI*”, asegurando que atributos como espaciados, márgenes, tipografías, colores y efectos se implementen de forma consistente **hasta el último píxel** ⁶. Esto implica que botones, iconos, campos de formulario, etc., deben respetar las dimensiones y posiciones definidas en el diseño en todas sus variaciones de estado y en las diferentes vistas (por ejemplo, en cada breakpoint de responsive design) ⁷. Al aplicar técnicas de Pixel Perfect, el diseñador verifica minuciosamente que no existan desviaciones: por ejemplo, usar herramientas de inspección en Figma/Sketch para comparar valores de tamaños y posiciones, o exportar especificaciones detalladas para desarrollo.

Un diseño Pixel Perfect **maximiza la claridad y profesionalismo** de la interfaz, transmitiendo calidad. No obstante, lograrlo demanda atención y tiempo: hay que equilibrar la búsqueda de la perfección con la eficiencia. En contextos profesionales actuales, se promueve un Pixel Perfect pragmático, apoyándose en sistemas de diseño predefinidos (p. ej. Material Design, Carbon de IBM, etc.) que proveen componentes ya armonizados ⁸. Esto ahorra trabajo repetitivo y garantiza cumplir estándares de la industria desde el inicio. Además, la **consistencia de marca** es otro principio: un prototipo de alta fidelidad refleja fielmente la identidad visual establecida (colores corporativos, estilo de iconografía, tono general), de modo que cada pantalla luce parte del mismo producto ⁹. En síntesis, “**pixel perfect**” significa que el producto digital se ve pulido y coherente en cada detalle, tal como fue diseñado, cumpliendo con las expectativas de calidad del usuario y de los stakeholders de negocio.

Uso de Grillas de Diseño y Técnicas *Pixel Perfect*

Una de las técnicas esenciales para lograr *Pixel Perfect* es el uso de **grillas de construcción**. Las grillas proporcionan una estructura subyacente para alinear y dimensionar elementos de manera uniforme. En diseño de interfaces es común trabajar con una **grilla base de 8px**, lo que significa que todas las medidas de espaciado y tamaños de elementos se hacen en múltiplos de 8 píxeles (o subdivisiones como 4px para detalles más finos) ¹⁰. Esta regla, conocida como *8pt grid*, simplifica las decisiones de diseño y asegura que los componentes “encajen” visualmente entre sí. Por ejemplo, después de configurar una retícula de 8px en la herramienta de diseño, cada elemento colocado se desplaza en incrementos de 8px, manteniendo el ritmo visual consistente ¹¹. **Material Design** de Google recomienda de hecho alinear la mayoría de las mediciones a una retícula de 8dp para lograr layouts balanceados, tanto en el espaciado interno de componentes como en la separación general del diseño ¹².

Además de la retícula de espaciado, los diseños web utilizan típicamente **grillas de columnas** (frecuentemente de 12 columnas) para la distribución horizontal. Estas grillas de columnas, combinadas con márgenes y *gutters* (espacios entre columnas) consistentes, ayudan a que la interfaz sea responsive y mantenga proporciones armoniosas en distintas resoluciones. **Cómo crear una grilla en base a un número base**: se elige un módulo base (8px es muy usado por su factor común con 4 y 2) y se definen líneas guía verticales y horizontales espaciadas según ese módulo. Todos los elementos deben “ajustarse” a estas guías. Por ejemplo, si el padding interno de un botón es de 16px (2 módulos de 8px) y la fuente tiene un tamaño que también se alinea con la cuadrícula, el resultado se ve limpio y profesional. El uso disciplinado de la grilla garantiza que nada quede “fuera de lugar”, logrando esa precisión *Pixel Perfect*.

Diseños cómodos para el usuario: Trabajar con grillas también contribuye a la comodidad visual y usabilidad. Un layout bien estructurado evita extremos de distribución que perjudiquen la lectura o la interacción. Por ejemplo, limitar el ancho de los bloques de texto para mantener una longitud de línea legible (aproximadamente 50–80 caracteres por línea es lo recomendado) mejora la experiencia de lectura del usuario ¹³. Estudios tipográficos sugieren que tamaños de fuente de 14–16 px para párrafos, con un interlineado ~1.4 veces el tamaño de fuente, y longitudes de línea alrededor de 85 caracteres maximizan la legibilidad en pantalla ¹⁴. Un diseño “cómodo” considera estos principios: no extender líneas de texto demasiado largas en monitores anchos (mejor usar márgenes o múltiples columnas), proveer suficiente espacio en blanco (*white space*) para no abrumar la vista, y asegurar que los elementos táctiles en móviles tengan un tamaño adecuado para dedos. En resumen, la grilla y las guías de diseño no solo sirven para alinear elementos *pixel perfect*, sino que también ayudan a lograr **ergonomía visual**, evitando una experiencia incómoda. Un ejemplo de mala práctica sería un diseño fluido sin límites: en pantallas muy grandes las imágenes se estiran demasiado y en móviles el texto se vuelve ilegible ¹⁵, afectando negativamente la usabilidad ¹⁶. Por eso, incluso en un enfoque responsive, se suelen establecer anchos máximos para el contenido y puntos de quiebre donde la maquetación cambia, garantizando que el diseño se mantenga agradable a la vista y fácil de usar en cada tamaño.

Tipos de Layouts: Líquido, Adaptativo y Responsivo

Al diseñar un producto digital de alta fidelidad, es fundamental contemplar **cómo el layout se adaptará a distintos dispositivos**. Existen principalmente tres enfoques de diseño web multipantalla ¹⁷:

- **Diseño líquido o fluido**: Usa unidades relativas (porcentajes, em) en lugar de píxeles para que los elementos se expandan o contraigan según el ancho disponible. Esto hace que la interfaz

llene todo el espacio de pantalla en cualquier resolución. Si bien en tamaños medianos (ej. tablets, laptops) puede verse aceptable, en pantallas muy grandes o muy pequeñas surgen problemas: en monitores enormes el contenido puede dispersarse demasiado (imágenes excesivamente estiradas), y en móviles puede resultar en textos extremadamente pequeños o layouts desproporcionados ¹⁸. Aunque se pueden poner límites (anchos máximos/mínimos), un diseño puramente líquido tiende a perjudicar la usabilidad (p. ej. líneas de texto demasiado largas dificultan la lectura) ¹⁹ ¹⁶. Debido a estas dificultades de ergonomía, el diseño completamente líquido **no se suele recomendar** si se busca una experiencia óptima.

- **Diseño adaptativo (Adaptive Web Design):** Plantea **plantillas estáticas** definidas para distintos rangos de tamaño de pantalla. En lugar de un layout fluido continuo, el adaptativo define **puntos de quiebre** específicos: por ejemplo, se puede diseñar una versión de la página para escritorio, otra para tablet y otra para móvil. Cuando el usuario accede, el sistema detecta el dispositivo o resolución y carga la plantilla correspondiente ²⁰. Esto garantiza presentaciones ajustadas a ciertas categorías de dispositivo, pero fuera de esos rangos la interfaz no cambia hasta alcanzar el siguiente breakpoint. Una ventaja es que permite un control fino del diseño en cada tamaño predeterminado y puede requerir menos esfuerzo que planificar todos los tamaños posibles. Sin embargo, tiene **contras**: suele ofrecer una experiencia menos fluida en tamaños intermedios y, en móviles especialmente, a veces muestra una versión reducida del contenido (omitido o simplificado) ²¹, lo cual puede afectar la usabilidad pues el usuario móvil recibe menos funciones o información. Hoy en día, con el uso móvil en aumento, es arriesgado tratar la versión móvil como "segunda" – lo ideal es darle igual importancia que a la de escritorio.
- **Diseño responsivo (Responsive Web Design):** Es el enfoque más moderno y recomendado. Un diseño responsivo se adapta **flexiblemente** a cualquier tamaño de pantalla, reordenando y escalando los elementos para brindar una experiencia equivalente en resoluciones bajas, altas y formatos diversos ²². En vez de plantillas completamente distintas, el contenido se estructura en bloques o módulos que pueden reorganizarse y cambiar de estilo según reglas definidas para distintos breakpoints. Se utilizan combinaciones de consultas de media (*media queries* CSS) y unidades flexibles para que la presentación cambie gradualmente. Por ejemplo, se pueden definir reglas para que a cierto ancho la navegación pase de horizontal (menú en barra) a vertical (menú tipo hamburguesa), o que una grilla de 4 columnas en desktop pase a 2 columnas en tablet y a 1 columna en móvil. Importante: **todos los contenidos permanecen disponibles** sin importar el dispositivo; la información es la misma, solo que su disposición se optimiza según las características de la pantalla (ancho, orientación, densidad de píxel) ²³ ²⁴. El resultado es que el usuario ve una interfaz adaptada en cada dispositivo, pero con todas las funciones y contenido intactos, y una apariencia coherente con la marca. Este método suele brindar la **mejor experiencia de usuario**, ya que *"adapta los contenidos de forma comprensiva"* y considera múltiples escenarios de uso ²⁵. También conlleva beneficios técnicos: una sola base de código para todos los dispositivos (más fácil de mantener que versiones separadas) y mejoras en SEO al servir un sitio unificado móvil+desktop. Por estas razones, el diseño responsivo es el estándar actual para productos digitales de calidad.

En la práctica, los diseños reales pueden combinar enfoques. Por ejemplo, se puede implementar un diseño principalmente **responsivo**, pero con ciertos elementos adaptativos (p. ej. cargar una versión distinta de una imagen en móvil vs desktop). Para lograr un buen diseño responsivo Pixel Perfect, es importante planificar desde el inicio la estructura pensando en **breakpoints**.

Diseño Responsivo y Breakpoints (Puntos de quiebre)

Los **breakpoints** son puntos específicos en los que el diseño “rompe” o cambia su disposición para ajustarse mejor al espacio disponible ²⁶. Normalmente se definen por anchos de pantalla. Por ejemplo, un diseño puede tener un breakpoint alrededor de 768px (típico ancho de tablet en vertical) para reconfigurar la interfaz de formato de móvil a uno de tablet, y otro en 1200px para una vista de escritorio amplia. Según una clasificación común, los breakpoints estándar serían aproximadamente:

- *Móviles verticales*: ~320px a 575px de ancho
- *Móviles horizontales / pantallas pequeñas*: ~576px a 767px
- *Tablets (vertical/horizontal)*: ~768px a 991px
- *Escritorio estándar*: ~992px a 1199px
- *Escritorio ancho / monitores grandes*: 1200px o más ²⁷

(Nota: los valores exactos pueden variar, pero estos rangos sirven de guía de referencia.) En esos rangos, la maquetación debe reacomodarse.

¿Por qué importan los breakpoints? Porque garantizan que el diseño se vea y funcione bien en todos los dispositivos, evitando situaciones como menús que se rompen, textos que desbordan o imágenes deformadas en ciertos tamaños ²⁸. Usar breakpoints permite diseñar con un enfoque *mobile-first*: es decir, se diseña primero la versión móvil (pantalla pequeña) y luego se van añadiendo mejoras para pantallas más grandes conforme se pasan puntos de quiebre ²⁹. Esto suele producir interfaces más eficientes, porque asegurar la usabilidad en móvil (el caso más limitado) sienta una base sólida, y después las pantallas grandes pueden aprovechar espacio extra para distribuir mejor el contenido. Además, Google premia en SEO a los sitios mobile-friendly, por lo que un diseño responsive bien hecho (con breakpoints apropiados) ayuda en el posicionamiento ³⁰.

Buenas prácticas con breakpoints: No usar más de los necesarios – generalmente cubrir móvil, tablet y desktop es suficiente para la mayoría de productos, con quizás algún ajuste extra para “phablets” o pantallas muy grandes. Cada nuevo breakpoint añade complejidad, así que deben justificarse por un cambio notable en el diseño. También es útil emplear unidades relativas (% , em , rem) en combinación con breakpoints, para que la adaptación sea más suave. En diseño Pixel Perfect, el equipo debe preparar maquetas de alta fidelidad **para cada breakpoint principal**. Esto significa que, por ejemplo, se entreguen versiones del diseño para móvil, para tablet y para desktop, todas con nivel de detalle Pixel Perfect. De esta manera se cubren las “pantallas estáticas considerando breakpoints para los diferentes dispositivos”, como indica la práctica profesional. Cada diseño adaptado muestra cómo los componentes deben reorganizarse o redimensionarse, evitando ambigüedades para los desarrolladores. Por ejemplo, un **prototipo de pantallas estáticas** incluiría: pantallas clave en vista móvil (ancho ~375px), las mismas pantallas en vista tablet (~768px) y en escritorio (~1440px). Así se documenta visualmente el comportamiento responsive.

Finalmente, probar el diseño con herramientas o dispositivos reales es crucial: un diseño puede lucir bien en las maquetas, pero verificar en un teléfono físico o redimensionando el navegador confirma que los breakpoints funcionan correctamente y que la experiencia es consistente. En síntesis, incorporar breakpoints de manera adecuada asegura un **diseño responsive Pixel Perfect**, donde cada versión de la interfaz (móvil, tablet, desktop) está cuidadosamente diseñada y alineada con los estándares de la industria.

Prototipado Interactivo y Feedback de la Interfaz

Lograr alta fidelidad no solo atañe a lo visual estático, sino también a la **interacción**. Un prototipo de alta fidelidad debe ser preferiblemente **interactivo**, es decir, permitir la simulación del flujo de uso tal como un usuario lo experimentaría. A diferencia de un mockup estático (que es solo una imagen o diseño sin respuesta a la entrada del usuario), un **prototipo interactivo** incluye elementos clicables o manipulables que muestran el comportamiento del producto ³¹ ³². Por ejemplo, en un prototipo interactivo se puede hacer clic en un botón de “Menú” y ver cómo se despliega la navegación, o navegar de una pantalla de inicio a una pantalla de detalle mediante una transición animada. Esto resulta valioso para entender la *UX* antes de escribir código: se pueden realizar pruebas de usabilidad con usuarios, detectar confusiones en los flujos y evaluar microinteracciones (como animaciones de botones al pulsarlos) en un entorno realista.

Prototipos estáticos vs. interactivos: Un *mockup* de alta fidelidad tradicional muestra la apariencia final con precisión pixelada, pero **no reacciona**; en cambio, *un prototipo siempre es interactivo*, mostrando la funcionalidad y la experiencia de usuario que tendrá el diseño ³². La fidelidad visual de ambos puede ser similar (de hecho un prototipo de alta fidelidad luce idéntico al mockup), pero el prototipo requiere trabajo adicional para definir las interacciones. Se debe planificar qué pantallas conecta cada acción del usuario, qué transiciones o animaciones ocurren, y qué condiciones lógicas aplicar (por ejemplo, simulaciones de errores o de validaciones simples). Herramientas modernas de diseño (como Figma, Adobe XD, etc.) permiten construir estas interacciones sin código, mediante enlaces entre pantallas y definiciones de animaciones. Un buen prototipo interactivo de alta fidelidad brinda “*la ilusión*” de usar una aplicación casi real: el usuario puede clicar, deslizar, y recibir respuestas en la interfaz.

Al elaborar un prototipo interactivo **siguiendo buenas prácticas**, se incorporan elementos interactivos suficientes para **describir el comportamiento** del producto de manera coherente. Esto implica: incluir todos los estados relevantes de los componentes UI, mostrar cómo aparecen y desaparecen elementos (transiciones), y considerar la respuesta del sistema a las acciones del usuario. Por ejemplo, si el usuario realiza una acción crítica (como enviar un formulario), el prototipo debería mostrar un estado de carga o una pantalla de resultado, simulando qué ocurriría en el producto real. De esta forma se responde a la pregunta: “¿Qué esperan los usuarios luego de realizar acciones específicas?”. En general, los usuarios esperan **retroalimentación inmediata**; cualquier acción (un clic, un gesto) debería generar algún indicio de que el sistema la recibió ³³ ³⁴. Por tanto, diseñamos los prototipos para que den esa respuesta visual o sonora dentro de un **tiempo razonable** (idealmente instantáneo).

Estados de interacción y retroalimentación visual

Un componente interactivo (botón, campo de texto, etc.) suele tener múltiples **estados**: por ejemplo, un botón puede mostrarse en estado por defecto, con un estilo diferente al pasar el cursor por encima (*hover*), otro al ser presionado (*active*), un estado deshabilitado (grisado) cuando no está disponible, etc. Incluir estos estados en el diseño es importante para adelantar cómo reaccionará la UI. Los estados no aplican solo a botones; cualquier elemento con el que el usuario interactúe tiene que indicar su estado actual: enlaces visitados cambian de color, menús desplegables indican cuál opción está seleccionada, pestañas activas vs. inactivas, etc.

Además de los estados estáticos de los componentes, debemos prever **estados de resultado** tras las acciones del usuario. Por ejemplo, supongamos que en una aplicación de tareas el usuario pulsa “Guardar”. El diseño debe contemplar qué ocurre después: podría mostrarse un mensaje de éxito (“¡Tarea guardada exitosamente!”), o en caso de error, un mensaje de error indicando el problema.

También se suelen incluir animaciones sutiles como indicadores de progreso (un *spinner* girando mientras se guarda la tarea) para comunicar que la acción está en proceso. Según las buenas prácticas de UX, es esencial diseñar para una variedad de posibles estados, no solo el “camino feliz” donde todo sale bien ³⁵. **Diseñar estados de error, éxito, carga, vacíos, etc., guía al usuario** a lo largo de su recorrido y evita confusión ³⁶. Por ejemplo, startups y equipos novatos a veces olvidan los estados de error y enfocan el prototipo solo en el flujo ideal, pero para ofrecer la mejor experiencia hay que preparar la interfaz para esos “baches en el camino” con mensajes claros ³⁷.

Un conjunto mínimo de estados que conviene contemplar es: estado *por defecto* (lo normal, sin interacción), estado *hover/activo* (cuando el usuario interactúa, en entornos con cursor), estado de *proceso* o carga (mientras una acción tarda en completarse), estado de *éxito* (acción completada correctamente), estado de *error* (hubo un problema), y estado *vacío* ³⁸ ³⁹. Incorporar estos estados en el prototipo ayuda a *describir el comportamiento* completo del producto. Por ejemplo, diseñar una pantalla de “éxito” tras enviar un formulario, o un mensaje de error inline debajo de un campo de texto si la validación falla. De esta forma, los usuarios de prueba (o los stakeholders) pueden entender qué feedback recibirán en cada situación, y los desarrolladores tendrán claro qué deben implementar para cada caso.

Diseño de *Empty States*

Un caso especial de estado de la interfaz es el **estado vacío** (*empty state*). Un *empty state* ocurre cuando una pantalla o sección no tiene datos que mostrar. Puede ser la primera vez que el usuario abre la aplicación (sin contenido aún), o una lista donde se eliminaron todos los elementos, o una búsqueda que no arrojó resultados. En lugar de presentar simplemente un hueco vacío, las buenas prácticas indican que **diseñemos cuidadosamente estas pantallas vacías** para aprovecharlas en favor de la experiencia. De hecho, un empty state bien pensado puede mejorar la UX dando contexto y próximo paso al usuario ⁴⁰.

Por ejemplo, imagina que un usuario abre por primera vez una app de tareas y no hay ninguna tarea creada. Un diseño descuidado mostraría una pantalla blanca o un texto como “No hay tareas” y nada más. En cambio, un buen diseño de empty state **orienta al usuario**: podría mostrar un mensaje de bienvenida del tipo “Aquí verás tus tareas pendientes. Crea tu primera tarea para empezar” junto a un botón destacado “Añadir tarea”. Así, el usuario entiende qué falta (no hay datos porque es nuevo) y qué acción tomar a continuación ⁴⁰ ⁴¹. Otro ejemplo: si en un e-commerce una búsqueda no devuelve productos, en lugar de un espacio vacío se puede informar “No encontramos resultados para tu búsqueda” y sugerir al usuario revisar la ortografía o explorar categorías relacionadas. Esto evita que el usuario se sienta perdido o frustrado.

En resumen, el diseño de empty states debe cumplir con **tres funciones**: informar *qué ocurre* (por qué está vacío), proveer *orientación o acciones* (qué puede hacer el usuario a continuación) y, si es posible, *mantener el tono de la marca* para no romper la experiencia. Puede incluir ilustraciones o un toque de humor/delight, siempre que no distraiga del mensaje principal. Por ejemplo, muchas aplicaciones añaden un icono amable o una ilustración en sus empty states para reforzar el mensaje y dar calidez. También es una oportunidad para **personalizar** la experiencia: una app de fitness podría mostrar un dibujo motivacional y un mensaje “Aquí verás tu progreso. ¡Empieza tu primer entrenamiento!” en el estado vacío, acorde a su estilo ⁴². Asimismo, los empty states pueden **educar al usuario** sobre requisitos del sistema: si el usuario intenta acceder a una sección que requiere iniciar sesión, en lugar de un fallo silencioso, se muestra una pantalla vacía indicando “Debes ingresar a tu cuenta para ver esto” con un botón de Login ⁴¹.

Cuidar los empty states es importante porque, de lo contrario, pantallas en blanco pueden llevar a confusión, incertidumbre y abandono por parte del usuario ⁴³. Muchos diseños tradicionales pasaban por alto estos estados (al considerarlos casos extremos o poco frecuentes), pero en la actualidad se reconocen como **momentos clave** para mantener la confianza del usuario durante toda la interacción ⁴⁴ ⁴⁵. Un empty state bien diseñado convierte una situación potencialmente negativa (ausencia de contenido) en una experiencia constructiva, comunicando estado del sistema y ayudando al usuario a seguir involucrado con el producto ⁴⁶. En términos de Pixel Perfect, estos estados deben diseñarse con el mismo nivel de detalle visual que cualquier otra pantalla, asegurando que encajen estilísticamente con el resto de la interfaz.

Tipografías: Webfonts vs. Fuentes del sistema

Otro aspecto que influye en la experiencia de un prototipo de alta fidelidad es la elección tipográfica, en particular si usamos **web fonts (fuentes externas)** o **system fonts (fuentes del sistema)**. Una *webfont* es una tipografía personalizada que se carga junto con la página (por ejemplo, a través de Google Fonts o archivos alojados). Por otro lado, una *fente de sistema* es aquella ya instalada en el dispositivo del usuario (por ejemplo, Arial, Times New Roman, o las fuentes nativas del sistema operativo como San Francisco en iOS, Roboto en Android, Segoe UI en Windows, etc.).

Usar **fuentes web** tiene la ventaja de permitirnos integrar la identidad de marca con una tipografía específica y consistente en todos los dispositivos – aporta personalidad y diferenciación visual. Sin embargo, conlleva **desventajas** técnicas: la fuente debe descargarse antes de mostrarse, lo cual puede **ralentizar el rendimiento** del sitio o aplicación, especialmente con conexiones lentas ⁴⁷. Durante esa descarga, es posible que el usuario vea texto invisible o en una fuente por defecto (fenómeno conocido como *flash of invisible text* o *flash of unstyled text*). Esto puede causar saltos de maquetación cuando la fuente termina de cargar y el texto se re-renderiza con el nuevo tipo ⁴⁸. Además, las webfonts suelen implicar depender de servidores externos (por ejemplo, las Google Fonts se cargan desde servidores de Google), lo que introduce una dependencia: si ese servidor falla o está bloqueado, la fuente no cargará ⁴⁷. También existen consideraciones de privacidad y legalidad (en algunos lugares se debate si la carga de fuentes externas podría violar ciertas normativas al exponer datos de usuario) ⁴⁹.

Por el contrario, las **fuentes del sistema** prácticamente **no tienen costo de rendimiento**: al ya estar instaladas localmente en el dispositivo, no requieren ninguna descarga ⁵⁰. Esto significa que el texto aparece inmediatamente, sin retrasos ni cambios de estilo inesperados (evitando los “layout shifts” causados por fuentes tardías) ⁵¹ ⁵⁰. También eliminan la dependencia de terceros – la interfaz no depende de la disponibilidad de un CDN de fuentes – y aseguran consistencia con el entorno nativo (por ejemplo, usar la fuente por defecto del sistema operativo puede hacer que la app web se sienta más “integrada” en esa plataforma). Por estas razones, muchos sistemas de diseño priorizan hoy usar *font stacks* de fuentes nativas (combos de fuentes del sistema de distintos OS) para interfaces donde la velocidad y accesibilidad son críticas. Un ejemplo es la pila de fuentes `-apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, ...` que selecciona automáticamente la fuente UI nativa según el sistema del usuario, dando una apariencia adecuada en cada uno.

La **desventaja** de las fuentes de sistema es la pérdida de unicidad visual – estás limitado a tipografías comunes que quizá no comunican plenamente la identidad de marca. Sin embargo, cabe señalar que hay muchas opciones de system fonts modernas (por ejemplo, San Francisco, Roboto, etc. son bastante neutras y legibles) y se puede lograr una buena estética aprovechándolas. En resumen, es un **compromiso entre personalidad y rendimiento**: si la identidad de marca lo requiere, se empleará una webfont custom, asegurándose de optimizar su carga (por ejemplo, usando métodos como `preload` o formatos WOFF2 comprimidos). Pero si la prioridad es máxima velocidad y compatibilidad, usar fuentes del sistema puede ser preferible, ya que mejora el rendimiento, evita dependencias externas y

elimina riesgos de saltos de maquetación o problemas entre navegadores ⁵⁰. En cualquier caso, en un prototipo Pixel Perfect se debe especificar claramente la tipografía elegida, ya sea webfont o sistema, para que en desarrollo se replique correctamente.

Diseño de Interacción en Productos Digitales

Lograr una excelente experiencia usuaria no solo implica precisión visual, sino también entender **cómo interactúan los usuarios con el producto** en distintos contextos. El *diseño de interacción* abarca principios que van desde considerar los diferentes dispositivos de acceso, los gestos que empleamos, hasta conceptos de usabilidad como affordances y retroalimentación. A continuación, profundizamos en estos aspectos:

Interacción en múltiples dispositivos (desktop vs móvil)

Los usuarios hoy acceden a productos digitales mediante diversos dispositivos: PCs de escritorio con mouse y teclado, laptops con trackpad, tablets táctiles, smartphones, e incluso dispositivos emergentes como relojes inteligentes o interfaces de voz. Un diseñador debe tener en cuenta cómo varía la interacción en cada caso para proporcionar una experiencia consistente y agradable.

- **Entornos de escritorio (PC/Laptop):** Aquí la interacción principal es a través de **mouse/trackpad** y teclado. Esto permite acciones precisas (el cursor puede apuntar a pequeños controles) y además introduce el concepto de *hover* – el cursor puede posarse sobre elementos sin hacer clic. Muchas interfaces de escritorio aprovechan el *hover* para mostrar información adicional (como tooltips, menús desplegables al pasar el mouse, efectos de resalto en botones, etc.). El teclado también permite atajos y navegación focal (por ejemplo, moverse con Tab entre campos de formulario). El diseñador de interacción en desktop debe considerar estas capacidades: por ejemplo, asegurarse de que haya estados *hover* diseñados para botones o links, y que la secuencia de tabulación en formularios siga un orden lógico. También el *scroll* con rueda o gestos en trackpad es un medio fundamental: las páginas tienden a ser más verticales en desktop esperando que el usuario despliegue contenido con la rueda del mouse o el scroll.
- **Entornos móviles y táctiles:** En smartphones y tablets la interacción es **directamente táctil** – el usuario toca la pantalla con los dedos. Esto elimina la posibilidad de estados *hover* (no hay concepto de “pasar el cursor” con el dedo; el contacto es siempre un tap directo). Por tanto, las interfaces móviles deben diseñarse sin depender de *hover* para nada crítico. En su lugar, se suelen utilizar indicios táctiles (por ejemplo, un ligero cambio de color al tocar un botón, durante unos milisegundos, para indicar que fue registrado el tap). Además, los controles táctiles deben ser lo suficientemente grandes: se recomienda un mínimo de ~7-10 mm de tamaño (alrededor de 40px) para que un dedo pueda pulsarlos cómodamente sin equivocar el toque. Otro factor es la orientación: los móviles pueden rotar entre vertical y horizontal; un buen diseño considera ambas orientaciones (aunque suele optimizarse la vertical primero). Las pantallas táctiles modernas soportan **gestos multitouch**, lo que abre todo un abanico de interacciones adicionales (ver siguiente sección).

En un escenario de escritorio típico, el usuario interactúa con **clics** y movimiento de cursor en cada píxel de la pantalla, apoyado por elementos como scrollbars, punteros que cambian de forma (por ejemplo, una mano indicando que algo es clicable) y otros indicadores visuales ⁵². En un escenario táctil, el usuario emplea directamente sus dedos: *toques* para activar botones o seleccionar, *deslizamientos* para hacer scroll o navegar carruseles, *pellizcos* para hacer zoom, etc. La interfaz táctil debe ser **directa e intuitiva**, ya que el usuario literalmente “manipula” los elementos en la pantalla. Aquí aplica con fuerza

el principio de *feedback inmediato*: por ejemplo, al arrastrar un elemento, este debe moverse bajo el dedo para indicar que está siendo arrastrado; al hacer un *swipe* para refrescar contenido, se muestra una animación de pull-to-refresh, etc.

Un detalle interesante: en dispositivos táctiles públicos (como kioscos), a veces se incluye un cursor virtual para educar al usuario. Pero en general en móvil personal no hay cursor visible; la interfaz debe dar otras pistas (como breves animaciones o destellos) para confirmar acciones.

En conclusión, **diseñar para diferentes dispositivos** implica adaptar la interfaz a los métodos de entrada: aprovechar las fortalezas de cada entorno y mitigar sus limitaciones. Un buen prototipo de alta fidelidad debería contemplar estas diferencias. Por ejemplo, podría incluir pantallas con un *hover state* visible en la versión desktop, y ese mismo elemento quizás con un *tooltip* permanente o un diseño ligeramente distinto en la versión móvil (ya que en móvil el hover no existe). Así, garantizamos que la experiencia sea óptima tanto con ratón como con dedo.

Gestos e interacción táctil

En el mundo móvil y de pantallas táctiles, los **gestos** son una forma natural de interacción. Más allá de tocar (*tap*) para hacer clic, existen numerosos gestos multi-touch reconocidos universalmente y que los usuarios esperan poder usar en aplicaciones modernas ⁵³. Algunos de los gestos táctiles más comunes son:

- **Tap (toque):** Un toque simple con el dedo equivale al clic del ratón – se usa para activar botones, links, toggles, etc.
- **Double tap (doble toque):** Dos toques rápidos. A menudo utilizado para zoom (por ejemplo, en mapas o imágenes, doble tap acerca la vista) o para indicar “me gusta” en redes sociales (Instagram popularizó el doble tap en una foto para darle like).
- **Long press (toque prolongado):** Mantener el dedo presionando durante un breve tiempo. Suele abrir acciones secundarias o un menú contextual (similar a clic derecho). Por ejemplo, mantener pulsado un ícono en la pantalla de inicio de iOS activa el modo de reordenar o borrar apps.
- **Swipe (deslizar):** Arrastrar el dedo horizontal o verticalmente. Es un gesto muy versátil: se usa para navegar entre páginas (swipe lateral), para revelar opciones ocultas (swipe sobre un elemento de lista para mostrar botones de eliminar/editar), para refrescar contenido (swipe hacia abajo, *pull to refresh*), etc.
- **Scroll (desplazar):** Deslizar uno o dos dedos verticalmente para desplazarse por una lista o página larga. Normalmente un dedo basta para hacer scroll vertical, pero en ciertos contextos (como un mapa embebido dentro de una página) a veces se usan dos dedos para desplazarse, evitando conflictos con la propia página.
- **Pinch (pellizcar) para zoom:** Juntar dos dedos sobre la pantalla y separarlos (extendiendo) para acercar zoom, o lo inverso (pellizcar juntando dedos) para alejar zoom ⁵⁴. Este gesto es muy intuitivo para mapas, galerías de imágenes y cualquier contenido con zoom libre.
- **Rotate (girar):** Colocar dos dedos y girarlos en torno a un punto para rotar un elemento. Común en editores de fotos, por ejemplo.
- **Fling:** Es similar a un swipe pero *rápido*, lanzando un desplazamiento con inercia (muy común en scroll de listas: un flick de dedo hace que la lista se desplace y frene suavemente, simulando inercia física).

Estas interacciones táctiles enriquecen la experiencia, pero hay que **diseñarlas con cuidado**. Primero, asegurarse de que los gestos esenciales sean descubiertos fácilmente: un problema común de UX es ocultar funciones detrás de gestos que el usuario podría no descubrir (gestos invisibles). Por eso, se suelen emplear **indicadores visuales** o tutoriales iniciales: por ejemplo, puntitos que sugieren que se

puede deslizar una galería, flechas indicando scroll horizontal, o animaciones de rebote insinuando que “hay más” fuera de la vista. Segundo, mantener la consistencia con convenciones de plataforma: iOS y Android tienen algunos gestos nativos (como deslizar de borde de pantalla para volver atrás en iOS, o el gesto “atrás predictivo” nuevo en Android ⁵⁵). Una aplicación debe respetar esas pautas para no romper la expectativa del usuario.

En nuestro prototipo de alta fidelidad, **podemos simular gestos** en la medida de lo posible. Por ejemplo, Figma permite definir que al *drag* (arrastrar) ocurra cierta animación, o que al hacer *swipe-left* en un elemento de lista se navegue a X frame mostrando opciones. Si la herramienta no soporta algún gesto complejo (como pinch-to-zoom), al menos se debe documentar en la entrega que “esta imagen podrá hacer zoom con gesto de pellizcar” para que los desarrolladores lo implementen. Lo importante es reconocer los gestos principales que **esperan los usuarios en dispositivos táctiles** y diseñar la interacción para aprovecharlos: un gesto bien implementado puede hacer la navegación más fluida y la interfaz más minimalista (porque gestos sustituyen a botones en pantalla, ahorrando espacio) ⁵⁶.

Por último, cabe mencionar que existen también **gestos sin contacto (touchless)** en tecnologías emergentes (por ejemplo, cámaras que detectan movimientos de mano en el aire, o sensores como Kinect). Estos no son habituales en productos convencionales de web/móvil, pero forman parte del futuro de la interacción. En cualquier caso, el principio sigue siendo diseñar de forma *natural e intuitiva* – la interacción gestual debería sentirse como una extensión del movimiento humano cotidiano ⁵⁷.

Affordances y señales intuitivas

El concepto de **affordance** se refiere a las *posibilidades de acción* que sugiere un objeto por su mera apariencia. Fue introducido por el psicólogo James Gibson y popularizado en diseño de interacción por Don Norman. En términos simples, una affordance es aquello que le dice al usuario “¿qué puedo hacer con esto?”. En productos físicos, por ejemplo, el diseño de una tijera “invita” a meter los dedos en los agujeros y apretarla – su forma comunica su uso. En interfaces digitales, hablamos más propiamente de *affordances percibidas*: pistas visuales o sensoriales que indican cómo interactuar con un elemento.

Don Norman define las affordances (percibidas) como “*las características perceptibles del objeto que le confieren un aspecto intuitivo sobre cómo usarlo*” ⁵⁸. Es decir, el usuario, con solo ver el elemento en pantalla, debería intuir qué hacer con él, sin necesidad de instrucciones explícitas. Como dijo Norman: “*Cuando se aprovechan las [affordances], el usuario sabe qué hacer con solo mirar: no se necesitan imágenes, etiquetas o instrucciones*” ⁵⁹. Este es el ideal al que aspiramos en UX: una interfaz auto-explicativa gracias a sus pistas visuales.

Ejemplos comunes de affordances en UI digital incluyen: un botón con efecto de relieve y sombreados parece *clickable* (porque remite a un botón físico que sobresale esperando ser presionado), un texto subrayado azul sugiere que es un hipervínculo (por convención), un campo de texto con borde y relleno claro indica que se puede escribir allí, un control deslizable con forma de perilla indica que se puede arrastrar, etc. Incluso las animaciones pueden actuar como signifiers (señalizaciones): por ejemplo, un ícono de menú “hamburguesa” que se transforma en ícono de “X” al abrir el menú lateral está indicando al usuario que ese mismo botón ahora sirve para cerrar el menú.

Es útil distinguir tipos de affordances. Según el académico H. R. Hartson ⁶⁰ ⁶¹, hay al menos tres situaciones: - **Affordance perceptible**: la posibilidad de acción existe **y es evidente** para el usuario. Estas son las que buscamos: elementos cuyo diseño hace obvio cómo usarlos. Un botón estilizado como tal, un ícono de papelera en borrar, etc. El diseñador debe intentar que todas las acciones importantes tengan affordances perceptibles claras ⁶². - **Affordance oculta**: la acción es posible, pero **no se muestra directamente** al usuario. Puede ser intencional (por ejemplo, funciones avanzadas

escondidas tras un menú contextual o un gesto no anunciado) o por limitación de espacio. En estos casos, el usuario debe descubrir la función explorando. El diseñador puede facilitar ese descubrimiento con pistas sutiles – por ejemplo, un pequeño indicador gráfico, o feedback al interactuar parcialmente. Un caso típico es el scroll: sabemos que podemos desplazarnos aunque no haya un indicador explícito, pero a veces se agrega una flechita o contenido cortado para insinuar que hay más hacia abajo. - **Affordance falsa:** el usuario **percibe** una posibilidad de acción que en realidad **no existe** ⁶³. Este es el más peligroso, ya que lleva a confusión y frustración. Un ejemplo sería resaltar un texto como si fuera un enlace (subrayado y azul) cuando no lo es – el usuario intentará hacer clic sin resultado. O un elemento con apariencia de botón que no hace nada. Las affordances falsas deben evitarse completamente, pues rompen la confianza del usuario en la interfaz.

En diseño digital, nos apoyamos en patrones establecidos para asegurar affordances correctas. Muchas convenciones de UI (por ejemplo, íconos estándar como el engranaje para ajustes, el carrito para comprar, etc.) funcionan porque los usuarios ya *perciben* la acción asociada. Así, estamos aprovechando affordances aprendidas. Un buen diseñador equilibra innovar con mantener familiaridad: si creamos un control totalmente nuevo, debemos asegurarnos de diseñarle un affordance claro o explicarlo, de lo contrario nadie sabrá cómo usarlo. Por eso Norman luego enfatizó que en diseño nos importa sobre todo lo que el usuario **percibe** (más que las affordances “reales” del objeto) ⁶⁴.

En la práctica, para lograr interfaces intuitivas: - Utilizamos **metáforas visuales apropiadas** (p. ej., hacer que un ícono de chat parezca un globo de diálogo, lo cual sugiere “mensaje”). - Mantenemos **consistencia** en los elementos interactivos: todos los botones se ven similares, todos los enlaces tienen un estilo uniforme, etc., de modo que el usuario los reconozca como tales fácilmente. - Damos **feedback inmediato** cuando un affordance es probada: si algo parece clickeable y el usuario lo pulsa, debe ocurrir algo (cambiar de color, animarse) para confirmar que en efecto lo era. Si nada sucede, quizás era un affordance falso o mal diseñado. - Evitamos elementos decorativos que se confundan con elementos interactivos. Cada elemento que parece interactivo debería serlo, de lo contrario reconsiderar su estilo.

En un entorno de prototipo, podemos validar las affordances haciendo pruebas con usuarios: ¿saben ellos qué hacer en cada pantalla sin que se les diga? Si dudan sobre cómo avanzar, quizá falte un significar más claro (un botón más prominente, una etiqueta, una animación indicativa).

En resumen, las **affordances en productos digitales** tratan de que el usuario encuentre *natural* el uso de la interfaz, que “sepa” qué hacer al verla. Un diseño Pixel Perfect exitoso no es solo estética precisa, sino también usabilidad: todos los elementos deben comunicar su propósito. Cuando esto se logra, la experiencia se vuelve fluida, pues el usuario navega intuitivamente: sabe qué es clic, qué es deslizable, dónde puede ingresar datos, etc., sin tener que pensarlo dos veces. Esa es la meta final de combinar un **buen diseño visual con buen diseño de interacción**.

Herramientas de Prototipado Interactivo (Figma, InVision, Marvel)

Para llevar a la realidad estos prototipos de alta fidelidad interactivos, contamos con diversas **herramientas de prototipado** profesionales. Entre las más populares en la industria se encuentran **Figma**, **InVision** y **Marvel**, cada una con características particulares que facilitan el diseño centrado en la experiencia del usuario.

- **Figma:** Es una plataforma de diseño **colaborativa en la nube**, utilizable directamente desde el navegador web (o en aplicaciones nativas). Figma permite tanto el diseño de interfaces de alta

fidelidad como la creación de prototipos interactivos en un mismo lugar. Su filosofía es altamente colaborativa – de hecho, su eslogan es “Nothing great is made alone” – y múltiples diseñadores pueden trabajar simultáneamente en un archivo en tiempo real ⁶⁵. Entre sus características destacadas está **FigJam**, un tablero colaborativo para lluvia de ideas, diagramas y workshops, integrado en el ecosistema ⁶⁶. En Figma, el diseñador puede crear componentes reutilizables, aplicar una retícula (p. ej. 8px) fácilmente y luego definir interacciones entre frames: por ejemplo, “al hacer clic en botón X, navegar al frame Y con una animación de desplazamiento”. La herramienta ofrece animaciones avanzadas (movimientos suaves, arrastres, swaps de componentes, sobreposición de capas, etc.), permitiendo prototipos de alta fidelidad muy cercanos a la realidad. Otro punto fuerte es que Figma es **multiplataforma** y siempre actualizado, al ser web; también facilita la entrega a desarrollo con inspección de propiedades CSS. Su plan gratuito es bastante generoso, soportando hasta 3 proyectos en equipo sin costo ⁶⁷, lo que la hace muy accesible para estudiantes o pequeños equipos.

- **InVision:** InVision fue uno de los primeros grandes players en prototipado y colaboración de diseños estáticos. Se trata de una herramienta (originalmente web) pensada para **armar flujos interactivos** a partir de diseños estáticos importados (desde Sketch, Photoshop, etc.). En InVision uno sube las pantallas (imágenes) y luego añade hotspots para vincularlas, creando un prototipo clicable. Con el tiempo, InVision ha incorporado más funcionalidades de gestión de diseño. Por ejemplo, permite administrar componentes de diseño dentro de un tablero estilo Kanban – con columnas representando estados de progreso – para hacer seguimiento de qué partes del diseño están en desarrollo, en revisión o aprobadas ⁶⁸. También ofrece una potente integración con herramientas de trabajo colaborativo: Trello, Jira, Slack, etc., de modo que el flujo de diseño se comunique con el flujo de desarrollo ⁶⁹. Empresas grandes (LinkedIn, Twitter, Uber, Salesforce, por nombrar algunas) la han utilizado ampliamente, avalando su capacidad para proyectos de escala ⁷⁰. InVision es particularmente útil para compartir prototipos con clientes y recoger feedback directamente: los usuarios pueden hacer clic en el prototipo y dejar comentarios en puntos específicos de la pantalla. Hoy en día, con herramientas como Figma que integran diseño+prototipo, InVision se usa un poco menos para *diseñar* pero sigue siendo valioso para *presentar* y recopilar *feedback*. Su versión básica permite 1 proyecto gratuito, siendo de pago para más proyectos ⁷¹.

- **Marvel:** Marvel App es otra plataforma enfocada en el **prototipado rápido y simple**. Al igual que InVision, permite conectar pantallas estáticas en prototipos interactivos en cuestión de minutos, sin necesidad de código ⁷². Marvel destaca por su **facilidad de uso**: su curva de aprendizaje es muy baja, ideal para quienes no son diseñadores pero necesitan armar un mockup interactivo rápidamente. Es utilizada por empresas como BlaBlaCar, Stripe o BuzzFeed ⁷³. Marvel se presenta como una herramienta colaborativa centralizada: todos los recursos del proyecto (imágenes, pantallas, comentarios) viven en la plataforma, facilitando tener un *hub* único durante el diseño y testing ⁷⁴. Una función interesante de Marvel es que provee algunas capacidades de **pruebas de usuario** integradas y de transferencia a desarrollo (handoff), aunque en forma más simple que Figma o InVision. Para equipos pequeños o prototipos rápidos de apps móviles, Marvel resulta muy conveniente. Ofrece también un plan gratuito limitado a un proyecto único y luego planes de pago escalables para individuos, equipos y organizaciones ⁷⁵.

En la práctica profesional, la elección de herramienta depende de las necesidades del proyecto y la preferencia del equipo. **Figma** se ha convertido en un estándar de la industria por su poder todo-en-uno y colaboración en tiempo real (reemplazando en muchos casos la combinación Sketch+InVision de años previos). **InVision** aún se emplea en flujos donde los diseñadores usan otras herramientas de UI (Adobe XD, Sketch) pero quieren una plataforma común para prototipos y feedback. **Marvel** encuentra su nicho en prototipado veloz y accesible, especialmente en entornos educativos o de validación

temprana de ideas. Cabe mencionar que existen otras herramientas robustas como Adobe XD, Framer, ProtoPie, UXPin, entre otras, que ofrecen capacidades similares con distintos énfasis (animaciones avanzadas, interacción con datos, etc.) ⁷⁶ ⁷⁷. Lo importante es que todas estas herramientas nos permiten **materializar un prototipo interactivo de alta fidelidad** de manera eficiente, aplicando los principios que discutimos: podemos alinear los píxeles perfectamente, definir estilos coherentes, e incorporar interacciones y gestos – todo sin escribir código, sino mediante interfaces visuales diseñadas para creadores de experiencias.

En conclusión, la implementación de un prototipo interactivo de alta fidelidad requiere combinar un **diseño visual meticuloso (Pixel Perfect)** con un **diseño de interacción centrado en el usuario**. Al seguir los principios de Pixel Perfect, aseguramos que cada pantalla cumpla con estándares de la industria en detalle y consistencia visual. Al mismo tiempo, aplicando las buenas prácticas de interacción (considerando dispositivos, gestos, estados de feedback y affordances claras), garantizamos que ese hermoso diseño también sea usable e intuitivo. Con el apoyo de modernas herramientas de prototipado, podemos iterar y refinar tanto la apariencia como el comportamiento antes de entrar a desarrollo, lo que resulta en un producto final más pulido y una experiencia de usuario positiva. En síntesis, un prototipo de alta fidelidad bien hecho es **ver y sentir el futuro producto**: se ve *pixel-perfect* y se comporta de forma *user-friendly*, logrando el objetivo de enamorar a usuarios y stakeholders incluso antes de escribir la primera línea de código. ⁷⁸ ³⁶

¹ ² ³ ⁴ ⁵ ⁹ ⁷⁸ Diseño de Alta Fidelidad (Pixel Perfect): Viendo el Futuro de tu Web

<https://www.martewebsitebuilder.com/blog/disenio-de-alta-fidelidad-pixel-perfect-viendo-el-futuro-de-tu-web>

⁶ ⁷ ⁸ ³⁵ ³⁶ ³⁷ ³⁸ ³⁹ Pragmatic Pixel-Perfect Design: Quality vs. Speed | Toptal®

<https://www.toptal.com/designers/ui/pragmatic-pixel-perfect-design>

¹⁰ ¹¹ Migrar para crecer. La experiencia de diseñar una interfaz... | by Norali Nayla | Flux IT Thoughts | Medium

<https://medium.com/flux-it-thoughts/migrar-para-crecer-f171ce44c354>

¹² Understanding layout - Material Design 2

<https://m2.material.io/design/layout/understanding-layout.html>

¹³ ¹⁴ Web Fonts are Critical to the Online User Experience - Don't Hurt Your Reader's Eyes | IxDF

https://www.interaction-design.org/literature/article/web-fonts-are-critical-to-the-online-user-experience-don-t-hurt-your-reader-s-eyes?srsId=AfmBOOpK5cT84lqtX-Erk5l97WYPLpTaChsR-kj_2nE8e6fKcs1EaU8W

¹⁵ ¹⁶ ¹⁷ ¹⁸ ¹⁹ ²⁰ ²¹ ²² ²³ ²⁴ ²⁵ ¿Qué son los diseños web fluidos, adaptativos y responsivos?

<https://blog.ida.cl/disenio/diferencias-diseno-web-fluido-adaptativo-responsivo/>

²⁶ ²⁷ ²⁸ ²⁹ ³⁰ Breakpoints en diseño web responsive: Qué son y cómo utilizarlos correctamente

<https://tuestudioweb.com/breakpoints-en-diseno-web-responsive-que-son-y-como-utilizarlos-correctamente/>

³¹ ³² Mockups vs prototipos: Aclarando términos de diseño | Miro

<https://miro.com/es/mockup/mockup-vs-prototipo/>

³³ ³⁴ 10 Usability Heuristics for User Interface Design - NN/G

<https://www.nngroup.com/articles/ten-usability-heuristics/>

⁴⁰ ⁴¹ ⁴² UX/UI: Empty States - MARKETINGWOW.ES

<https://marketingwow.es/blog/ux-ui-empty-states/>

⁴³ ⁴⁴ ⁴⁵ ⁴⁶ Empty States – The Most Overlooked Aspect of UX | Toptal®

<https://www.toptal.com/designers/ux/empty-state-ux-design>

47 48 49 50 51 The Pitfalls of Web Fonts: Embracing the Benefits of System Fonts for Web Design - The Joomla Community Magazine

<https://magazine.joomla.org/all-issues/december-2023/the-pitfalls-of-web-fonts-embracing-the-benefits-of-system-fonts-for-web-design>

52 58 59 60 61 62 63 64 Affordance: Posibilidades que ofrece un objeto para saber usarlo - IDA Blog | UX y Marketing para vincular marcas y audiencias.

<https://blog.ida.cl/disenio/affordance-posibilidades-objeto-para-usarlo/>

53 54 57 Design Toolkit | Gesture

<https://design-toolkit.recursos.uoc.edu/es/gesture/>

55 Diseño del gesto atrás predictivo | Mobile - Android Developers

<https://developer.android.com/design/ui/mobile/guides/patterns/predictive-back?hl=es-419>

56 Interacción y patrones - Diseñando apps para móviles

<https://www.appdesignbook.com/es/contenidos/patrones-interaccion-moviles>

65 66 67 68 69 70 71 72 73 74 75 76 77 Prototyping Tools: las mejores herramientas de creación de prototipos para UX Designer

<https://blog.talentgarden.com/es/blog/design/las-mejores-herramientas-de-prototipos-ux-designer>